

面向 LLM 智能体的自动化提示注入攻击研究

吴丰艺

2026 年 6 月

1 立题依据及价值

1.1 研究背景与国内外研究现状

大语言模型 (Large Language Models, LLMs) 已从单纯的文本生成系统演进为能够在现实世界中执行操作的自主智能体。现代 LLM 智能体可与外部工具交互、访问邮件与日历、查询数据库、浏览网页并执行代码 [1, 2]，这一演进催生了个人助理管理日程与通信、企业自动化系统处理文档与协调工作流、客服代理访问后端系统解决查询等诸多应用 [3]。然而，能力提升的同时带来了显著的安全风险：当 LLM 不仅能生成文本，还能执行操作时，对抗性攻击的后果变得更加严重。能够发送邮件、转移资金或修改文件的智能体，与仅生成文本的聊天机器人相比，呈现出根本不同的威胁面。理解和评估这些漏洞对于 LLM 智能体的安全部署至关重要。

提示注入攻击 (Prompt Injection) 利用了 LLM 处理自然语言的基本特性。首先，指令与数据之间没有明确的边界 [4, 5]；其次，模型必须根据上下文理解来决定遵循哪些指令，而这一上下文敏感的指令遵循能力恰恰是智能体有效运作的关键。然而，同一能力也使智能体容易被操纵。攻击者将恶意指令嵌入到模型处理的内容中，由于模型无法仅根据来源或上下文可靠区分合法指令与对抗性指令，可能执行攻击者的命令。间接提示注入对 LLM 智能体构成了尤为严重的威胁 [4]，攻击者不直接与智能体交互，而是将恶意指令嵌入智能体在运行过程中检索的外部内容（如邮件、文档或网页），导致数据泄露、未授权操作和输出完整性受损等后果。

当前，关于 LLM 自动化攻击的研究主要集中在越狱 (Jailbreaking) 方面，即诱使已对齐模型生成被禁止内容。GCG[6]、AutoDAN[7]、PAIR[8] 和 TAP[9] 等方法已在越狱场景中被开发和广泛评估，在标准基准上实现了高成功率。相比之下，提示注入攻击在对抗性机器学习社区中受到的关注远少于越狱攻击。在提示注入文献内部，研究大多聚焦于直接提示注入场景与简单评估设置，多数工作依赖手工攻击或基本启发式策略 [10, 11]，而非自动化优化方法。评估场景通常限于简化任务（如带有恶意指令的文本摘要），在具有工具调用能力的智能体操作有状态环境的现实场景中，系统性的自动化攻击评估仍然匮乏 [12]。

1.1.1 基于越狱的 LLM 对抗攻击技术

越狱攻击旨在诱使已安全对齐的 LLM 生成被禁止内容，根据模型访问级别可分为白盒和黑盒两大类。

白盒优化方法。基于梯度的攻击利用对模型参数的白盒访问，通过直接微分优化对抗输入。Zou 等 [6] 提出 GCG (Greedy Coordinate Gradient) 算法，其核心思想是计算损失函数关于 one-

hot token 嵌入的梯度，在每个对抗位置选取梯度幅度最大的 k 个候选 token，通过批量前向传播评估每个候选替换的损失，选择使损失最小化的替换。该方法在白盒 Vicuna 模型上实现 88% 攻击成功率，且优化的对抗后缀对黑盒系统表现出显著可迁移性（GPT-3.5 达 87%、GPT-4 达 47%），揭示了 LLM 对抗性脆弱的跨架构普遍性。Geisler 等 [13] 引入投影梯度下降（PGD）方法攻击 RAG 系统，通过连续松弛重构离散 token 优化问题，在检索增强的生成管道中实现 10 倍计算加速下的可比成功率，并将攻击扩展到多文档检索场景。Wen 等 [14] 建立梯度引导离散 token 搜索的通用范式，形式化离散优化中的硬约束问题，为后续 GCG 等算法提供了理论框架与算法基础。Liu 等 [7] 提出 AutoDAN，采用遗传算法与分层交叉算子生成语义连贯、可解释的对抗提示。不同于 GCG 产生不可读的 token 序列，AutoDAN 通过在语义空间中搜索，在保持与 GCG 竞争性成功率的同时生成更低困惑度、更难被检测的提示。Andriushchenko 等 [15] 证明仅使用 logprob 反馈（无需梯度）的简单自适应攻击即可在 GPT-4、Claude 和 Gemini 等领先安全对齐模型上实现近 100% 成功率，揭示当前对齐方法的脆弱性，表明现有安全训练未能建立真正的安全边界。

黑盒优化方法。Mehrotra 等 [9] 提出 TAP (Tree of Attacks with Pruning) 算法，采用三模型架构：攻击者 LLM 负责根据反馈迭代生成越狱候选，评估器 LLM 判断目标模型响应是否包含有害内容并评分，目标 LLM 作为被攻击对象。TAP 通过树结构搜索（宽度 w 、深度 d ）探索多种攻击策略，利用 on-topic 分类器剪枝偏离目标方向的分支，在 GPT-4 上以平均 29 次查询实现 80–94% 成功率。Sitawarin 等 [16] 提出 PAL (Proxy-Guided Attack)，利用代理模型（如 Llama-2）上的 GCG 优化攻击后迁移至目标模型，结合智能重排序和多候选策略提高迁移成功率，首次系统研究了越狱攻击的代理辅助优化范式。Chen 等 [17] 提出 InstructZero，通过查询高效探索系统提示空间，在贝叶斯优化框架下，将黑盒越狱转化为连续提示嵌入空间上的优化问题，在无需梯度访问下实现与白盒方法竞争的性能。

1.1.2 基于提示注入的 LLM 智能体攻击技术

白盒优化方法。Pasquini 等 [18] 提出 Neural Exec，针对 RAG 系统中的延迟提示注入（deferred prompt injection），通过优化在检索条件下激活的“执行触发器”，不同于即时攻击，该触发器仅在特定检索上下文中激活恶意指令，在 RAG 系统上实现 87% 攻击成功率，揭示了检索增强机制引入的新攻击面。Fu 等 [19] 引入 Imprompter，将 GCG 优化应用于生成导致智能体误用工具的混淆提示，首次将 GCG 从越狱场景适配到智能体工具误用场景，通过优化 token 序列使智能体调用非预期工具或传递错误参数。Zhan 等 [20] 系统研究自适应攻击对智能体防御的威胁，通过修改 GCG 和 AutoDAN 显式考虑防御机制（如结构化分隔符和指令层级），在 StruQ、SecAlign 等多种防御范式下均实现超 50% 攻击成功率，证明当前防御在自适应攻击者面前的脆弱性。Wu 等 [21] 将对抗鲁棒性评估扩展到多模态 LM 智能体，在视觉-语言模型智能体上实现跨模态攻击，揭示智能体攻击面在多模态设置下显著扩大，恶意图像可与文本注入协同增强攻击效果。

黑盒优化方法。Liu 等 [22] 提出 HOUIYI 框架，将提示注入攻击系统化分解为三个组件：框架（framework，建立攻击上下文）、分隔符（separator，界定指令边界）和破坏器（disruptor，执行恶意指令），通过对这三个维度进行组合优化，在跨多个 LLM 应用（包括 GPT-4 和 Claude）上实现高成功率。Zhang 等 [23] 提出 G2PIA (Goal-guided Prompt Injection Attack)，结合理论攻击

原则（如权威伪装、紧急性框架等心理学原则）与 LLM 引导生成来产生多样化有效提示注入，通过自动化策略选择避免手工设计注入模板。Wang 等 [24] 提出基于蒙特卡洛树搜索（MCTS）的 AgentVigil，将提示注入攻击建模为序列决策问题，以选择动作类型和注入内容作为搜索节点，在 AgentDojo 基准上实现 71% 攻击成功率，代表当前智能体攻击的先进水平，但依赖手工设计的动作空间。Yu 等 [25] 提出 PromptFuzz，采用模板生成与 LLM 引导变异的两阶段模糊测试框架：第一阶段基于规则生成初始注入集合，第二阶段利用 LLM 对成功注入进行语义变异和增强，在多个智能体框架上实现攻击。

其他攻击方法。Labunets 等 [26] 利用微调 API 从闭源模型提取梯度等价信息进行优化攻击，绕过模型权重不可访问的限制，通过观察微调前后模型输出的差异反推梯度方向，在 API-only 设置下实现接近白盒的攻击效果。Shi 等 [27] 针对智能体推理循环中的工具选择决策实施两阶段优化攻击：第一阶段优化注入使智能体偏离原始推理路径，第二阶段优化注入使智能体执行攻击者指定的工具调用，揭示了智能体推理链的脆弱性。

1.1.3 LLM 智能体防御与评估技术

基于微调的防御。Chen 等 [28] 提出 StruQ，通过结构化分隔符（如特殊 token 标记系统指令、用户输入和数据区域的边界）强制指令-数据分离，并在该分隔符格式上微调模型仅遵循系统指令区域的内容，在手工攻击下实现低于 2% 攻击成功率，但在自适应 GCG 攻击下成功率仍可超 50%。Wallace 等 [29] 形式化指令层级（Instruction Hierarchy）概念，训练 LLM 在面对相互冲突的指令时优先遵循系统指令而非用户输入或外部数据中的指令，通过构建包含指令冲突的训练数据实现 30% 以上鲁棒性提升。Chen 等 [30] 提出 SecAlign，利用直接偏好优化（DPO）训练模型区分合法指令与注入内容，对每个训练样本构造“正确遵循用户指令”的正例和“误信注入内容”的负例，通过偏好学习使模型倾向正例行为，是首个将 GCG 攻击成功率降至 10% 以下的防御方法。

基于检测的防御。Zhu 等 [31] 提出 MELON (Masked Output Evaluation)，核心思想是对工具输出进行遮蔽后重新执行智能体——若遮蔽可疑内容后智能体行为发生显著变化，则判定存在注入攻击，在 GPT-4o 上实现 0.24% 攻击成功率，但增加了额外的推理成本。Hung 等 [32] 提出 AttentionTracker，利用注意力模式中的“分心效应”检测提示注入——当模型对注入内容的注意力权重异常高于正常数据时，表明可能存在注入，该方法无需修改模型即可部署，但对语义隐蔽的注入检测能力有限。Jacob 等 [33] 开发 PromptShield，训练专门的注入检测分类器，在 0.1% 假阳性率下实现 65.3% 真阳性率，兼顾了低误报与高检出。Liu 等 [34] 提出 DataSentinel，以博弈论方法进行极小极大优化训练检测器——将检测器与攻击者建模为零和博弈，检测器最小化最坏情况下的漏检率，在对抗自适应攻击时保持鲁棒。然而，Hackett 等 [35] 系统评估了主流护栏系统的鲁棒性，证明许多护栏系统可通过字符注入（如零宽字符、同形字替换）和编码操纵（如 Base64、Unicode 转义）绕过，揭示基于文本模式匹配的检测方法本质上的脆弱性。

架构与系统级防御。Wang 等 [36] 提出 CachePrune，基于特征归因的 KV 缓存神经元剪枝缓解提示注入——识别并剪除对注入内容响应贡献最大的注意力头对应的 KV 缓存条目，从计算图层面削弱注入对模型行为的影响。Zhong 等 [37] 提出 RTBAS (Run-Time Based Agent Security)，将信息流控制（IFC）应用于 LLM 智能体，为每个数据源分配机密性和完整性标签，运行时跟踪信息流确保高完整性指令不被低完整性数据污染。Debenedetti 等 [38] 提出从设计上击败提示注

入的架构方案 CaMeL，通过将智能体操作权限与数据流分离，确保外部数据仅作为信息输入而不能直接触发工具调用。Zverev 等 [39, 40] 从计算复杂性理论出发，证明 LLM 在计算上无法可靠区分指令与数据——将该问题归约为不可判定问题，为“完全防御提示注入是不可能的”提供了理论支撑。

评估框架。Debenedetti 等 [12] 提出 AgentDojo 框架，提供跨 4 个域（工作区、银行、旅行、Slack）的有状态智能体执行环境，包含 900+ 测试用例，通过确定性检查函数（验证工具调用名称和参数是否匹配攻击者目标）评估攻击成功率，而非简单的字符串匹配，是目前最接近真实部署的智能体安全评估平台。Zhan 等 [41] 提出 InjecAgent 基准，评估工具集成 LLM 智能体中的间接提示注入，包含 1054 个测试用例，在 ReAct 提示的 GPT-4 上报告 24% 攻击成功率，但评估场景相对简化。Fang 等 [42] 提出 FIDES，基于信息流控制的安全 AI 智能体方案——通过动态污点跟踪追踪机密性和完整性标签的传播，在 AgentDojo 上验证了该方案可在保证安全性的同时完成广泛的合法任务。

1.2 现有研究存在的问题

1. **越狱攻击方法难以直接迁移到提示注入场景。**现有的自动化对抗攻击方法（如 GCG、TAP）主要针对越狱场景设计，其优化目标和成功判定标准与提示注入攻击存在本质差异：越狱关注是否生成被禁止内容（可通过字符串匹配或 LLM 判断），而提示注入需要触发特定工具调用及正确参数（二元结果，迭代信号有限）。智能体的多轮交互、有状态环境和工具调用决策边界使攻击优化问题更加复杂，直接迁移的攻击成功率显著低于越狱场景。
2. **提示注入攻击评估缺乏系统性的自动化方法。**当前提示注入研究大多依赖手工攻击或简单模板，缺乏在真实智能体环境中进行自动化优化攻击评估的系统性方法。现有评估场景往往限于简化任务（如文本摘要中的恶意指令），而在具有工具调用能力的智能体操作有状态环境中的评估严重不足，无法充分测试智能体对自适应对抗者的鲁棒性。
3. **攻击的可迁移性与泛化能力不足。**在开源小模型上优化的攻击几乎完全无法迁移到先进闭源模型（如 GPT-5），成功率低于 2%。跨任务域和跨模型的攻击泛化仍面临根本性挑战：攻击优化的目标与真实部署目标之间存在对齐差距，单一输出序列的优化方式无法覆盖实际中的多种成功输出，且不同模型的工具调用行为和防御特性差异显著。

1.3 本研究的创新点

1. **面向 LLM 智能体间接提示注入的攻击目标形式化与适配。**将越狱攻击中的攻击目标从“生成被禁止内容”重新形式化为“触发特定工具调用并传递正确参数”的条件概率最大化问题，提出了肯定响应目标（Affirmative Response Target）与 LLM 翻译目标两种目标字符串构造策略，解决了越狱与提示注入场景间优化目标对齐的核心难题。
2. **通用攻击的跨场景联合优化与迁移性评估框架。**提出跨多任务域的通用攻击联合优化方法——通过在多场景训练集上平均梯度和损失来优化单一注入字符串（或可复用的前缀-后缀组件），并设计保留完全未见域的泛化性评估协议，首次系统性地研究了间接提示注入攻击在智能体场景中的跨任务泛化能力。

3. **面向智能体攻击的 Tokenization 验证与评估可靠性增强机制。**发现并提出上下文相关分词（context-dependent tokenization）导致的优化-部署编码不一致问题，设计 decode-reencode 验证过滤器确保 token 序列的编码-解码往返一致性；针对 LLM-as-judge 在智能体评估中准确率不足（如 Qwen 上仅约 50%）的问题，提出多级评估策略与可靠性测试机制，显著提升黑盒攻击的优化信号质量。

1.4 研究意义与价值

1.4.1 理论意义

1. 将越狱攻击中的白盒梯度优化（GCG）与黑盒迭代搜索（TAP）两种主流方法系统性迁移到间接提示注入场景，揭示自动化对抗方法从文本生成到动作执行场景迁移中的理论挑战与性能边界，丰富 LLM 智能体安全评估的方法论体系。
2. 构建通用攻击优化框架，研究在多任务、多场景联合优化下攻击的泛化机制，探索通用注入模式的存在性与可发现性，为理解 LLM 智能体的系统性脆弱点提供理论支撑。
3. 通过全面的消融实验与迁移性分析，揭示影响攻击成功的关键因素（注入结构、语义内容、初始化随机性、评估器可靠性等），为智能体安全防御的设计提供攻击侧的理论依据。

1.4.2 实用价值

1. 扩展 AgentDojo 框架以支持自动化攻击优化，集成 HuggingFace Transformers 库实现白盒梯度攻击，开发标准化攻击集成接口，为 LLM 智能体安全评估提供可直接使用的开源工具。
2. 在跨多域（工作区、银行、旅行、Slack）和多模型（Qwen3-4B、Gemma-3-4B、GPT-5）的 80 个任务对上进行全面实证评估，为工业界部署 LLM 智能体提供量化的安全风险评估参考。
3. 针对开源模型高达 45.2% 攻击成功率和 72.5% 安全阈值突破率的实验结果，为安全工程师设计防御策略、评估护栏系统有效性提供实证依据和攻击基线。

1.4.3 与国家及行业发展战略的契合度

1. 契合《新一代人工智能发展规划》中关于“人工智能安全”的战略部署，为 AI 系统的安全评估与风险防控提供关键技术支撑。
2. 响应《网络安全法》和《数据安全法》对关键信息基础设施安全保护的要求，为 LLM 智能体在金融、政务等敏感领域的安全部署提供攻击评估方法。
3. 服务于国家人工智能标准化总体组关于“人工智能可信特征”标准的研究，为 LLM 智能体鲁棒性与安全性评测提供方法论参考。

2 研究内容及方法

本研究围绕”如何将自动化对抗攻击方法从越狱场景迁移到 LLM 智能体间接提示注入场景”这一核心问题，以 AgentDojo 评估框架为基础平台，系统性地适配和评估 GCG（白盒梯度优化）与 TAP（黑盒迭代搜索）两大类攻击方法，并通过全面的消融实验与迁移性分析揭示影响攻击成功的关键因素。具体研究内容分为以下两个方面。

2.1 研究框架

本研究整体遵循”威胁建模、框架扩展、攻击适配、实验评估、结果分析”的研究路线，各环节逻辑自洽、紧密衔接。在威胁建模阶段，明确间接提示注入的攻击者能力边界与优化目标形式化；在框架扩展阶段，将 AgentDojo 与 HuggingFace Transformers 库集成，实现白盒梯度访问与标准化攻击接口；在攻击适配阶段，分别实现 GCG 和 TAP 在间接提示注入场景下的算法改造与优化策略；在实验评估阶段，在跨多域、多模型的 80 个任务对上进行全面实验；在结果分析阶段，通过消融实验和迁移性研究揭示攻击成功的关键影响因素。研究框架如图 1所示。



图 1: 面向 LLM 智能体的自动化提示注入攻击研究框架

2.2 研究点一：基于 GCG 白盒梯度优化的间接提示注入攻击

2.2.1 研究内容

围绕白盒梯度优化方法从越狱到间接提示注入场景的迁移问题，研究 GCG 算法在智能体工具调用环境中的适配与优化。首先，扩展 AgentDojo 框架以支持 HuggingFace Transformers 库集成，实现对开源模型（Qwen3-4B、Gemma-3-4B）的白盒梯度访问，解决模型特定聊天模板、工具调用格式和消息格式处理等工程挑战；其次，设计适应间接提示注入的优化目标函数，将攻击目标从“生成初始肯定响应”转变为“触发特定工具调用并传递正确参数”的二元判定，并开发 tokenization 验证机制确保优化与部署环境间的编码一致性；然后，实现单任务攻击与通用攻击两种优化配置：单任务攻击针对每个具体测试用例单独优化注入字符串，通用攻击则在多任务、多场景上联合优化单一注入字符串以发现广泛适用的漏洞模式；在此基础上，设计前缀注入、后缀注入和前缀 + 后缀组合等多种注入结构策略，探索注入位置对攻击效果的影响；同时，实现梯度引导优化与随机搜索的消融对照，评估梯度信号相对于随机搜索的实际贡献。

2.2.2 关键问题与拟采用方案

关键问题 1：优化目标与真实攻击目标之间的对齐差距 越狱场景中，攻击者仅需模型输出包含被禁止内容的文本片段，目标序列 y^* 可直接指定为有害文本（如炸弹制作步骤），优化成功即代表攻击成功。然而在间接提示注入场景中，攻击成功的判定发生了根本性变化，攻击者需要智能体执行特定的工具调用（如调用 `send_email` 向攻击者邮箱发送邮件、调用 `transfer_funds` 向指定账户转账），而非仅仅生成有害文本。这一转变带来三重对齐挑战：第一，目标序列的构造难度显著增加，工具调用需要严格遵循 JSON 格式规范（包含正确的工具名称、参数键和参数值），任何格式偏差（如缺少引号、参数名拼写错误）都会导致调用失败，而 JSON 格式的结构化 token（括号、引号、冒号）消耗大量目标长度，使优化更加困难；第二，最优目标形式不唯一，直接优化工具调用 JSON 可能过于刚硬（模型通常先以自然语言确认再调用工具），而优化自然语言肯定响应（如“I will send an email to attacker@example.com”）虽更符合模型输出分布，但无法保证后续必定执行正确工具调用，存在“口头承诺但不执行”的风险；第三，多步攻击存在延迟成功问题，部分攻击目标需要多轮工具调用才能完成（如先搜索文件再删除），而优化只能针对首轮输出，无法保证后续步骤的执行。

设 LLM 参数为 θ ，完整输入提示为 token 序列 $\mathbf{x} = [x_1, \dots, x_n]$ ，其中攻击者可控位置的子集为 $I \subset \{1, \dots, n\}$ ，可控部分记为 $\mathbf{x}_{\text{adv}}$ ，固定部分记为 $\mathbf{x}_{\text{fixed}}$ 。攻击者目标编码为目标输出序列 $y^* = [y_1^*, \dots, y_m^*]$ ，则单实例攻击优化问题形式化为：

$$\mathbf{x}_{\text{adv}}^* = \arg \max_{\mathbf{x}_{\text{adv}}} P_{\theta}(y^* | \mathbf{x}_{\text{fixed}}, \mathbf{x}_{\text{adv}}) \quad (1)$$

其中概率按自回归方式计算： $P_{\theta}(y^* | \mathbf{x}) = \prod_{i=1}^m P_{\theta}(y_i^* | \mathbf{x}, y_{<i}^*)$ 。在越狱场景中 y^* 为被禁止的文本片段，而在间接提示注入场景中 y^* 需要为包含正确工具名称和参数的 JSON 调用格式 `{"name": "f", "parameters": {...}}` 或自然语言肯定响应。拟通过设计目标字符串模板（包括工具调用 JSON 目标与 LLM 翻译的肯定响应目标），使用负对数似然损失 $\mathcal{L} = -\sum_{i=1}^m \log P_{\theta}(y_i^* | \mathbf{x}, y_{<i}^*)$ 进行优化。

关键问题 2：离散 token 优化的收敛不稳定性 GCG 的核心优化过程在离散 token 空间上进行，而智能体场景的提示上下文远比越狱场景复杂：系统提示包含详细的工具 schema 定义、多轮对话历史和角色标记，使得输入序列 $\mathbf{x}$ 的长度可达数千 token，优化面临更为严重的非凸性和收敛不稳定性问题。具体而言，不同随机种子初始化的优化运行可能产生截然不同的结果：某些种子迅速收敛至高成功率注入，而另一些种子在数百步迭代后仍停留在高损失区域。实验观察到 GCG 在 Qwen3-4B 上不同随机初始化的攻击成功率标准差可达 15% 以上，远高于越狱场景中的变异程度。这种不稳定性的根源在于三方面：第一，离散空间的梯度仅为连续松弛的近似，token embedding 的 one-hot 表示使梯度只能指示“哪些 token 位置值得尝试替换”，但无法保证替换方向确实降低损失；第二，贪心替换存在耦合效应，替换位置 j 的 token 可能使其他位置的最优选择发生跳变，导致贪心策略在各位置间振荡而非协同收敛；第三，上下文相关分词（context-dependent tokenization）引入编码不一致，许多 tokenizer 在解码时执行上下文敏感的子词合并，导致 token 序列在优化上下文中解码为字符串 s ，但在实际部署上下文中（插入完整提示后）解码为不同字符串 s' ，使优化结果在实际环境中失效。

具体而言，GCG 在第 t 步迭代中对每个对抗位置 $j \in I$ ，计算损失函数关于 one-hot token 表示的梯度 $\nabla_{e_{x_j}} \mathcal{L}$ ，选取梯度幅度最大的 k 个候选 token $\mathcal{C}_j = \text{TopK}(|\nabla_{e_{x_j}} \mathcal{L}|, k)$ ，然后对每个候选替换评估批量损失：

$$x_j^{(t+1)} = \arg \min_{c \in \mathcal{C}_j} \mathcal{L}(\mathbf{x}^{(t)}[j \leftarrow c]) \quad (2)$$

但由于 token 空间的离散性，梯度仅为连续松弛的近似，且替换单个 token 可能使其他位置的最优解发生跳变，导致优化在局部极小值间振荡。拟通过多种初始化策略（随机初始化、语义模板初始化）、增大批量搜索规模 k （典型值 256-512）以及通用攻击的多场景联合优化来缓解收敛不稳定问题。同时，开发 decode-reencode 验证机制：对优化生成的 token 序列执行解码-重编码校验，若 $\text{Encode}(\text{Decode}(\mathbf{x}_{\text{adv}})) \neq \mathbf{x}_{\text{adv}}$ 则将候选标记为无效（损失设为 $+\infty$ ），确保编码-解码往返一致性。

上述 GCG 优化流程如算法 1 所示。其中，第 7-8 行实现 ASCII 候选过滤以缩减搜索空间并保证注入字符串的鲁棒性；第 10-14 行实现 decode-reencode 验证机制，排除分词不一致的无效候选；第 17-23 行实现连续成功早停策略，避免瞬态匹配导致的过早终止。

关键问题 3：通用攻击的跨场景泛化能力 在实际部署中，LLM 智能体面临多样化的运行环境，同一银行助手可能处理不同用户的转账请求，同一邮件客户端可能对接不同的邮件服务商 API。因此，攻击者嵌入在网页或文档中的注入字符串往往只有一次机会被检索到，且必须在一个此前未见过的智能体配置下生效。通用攻击的核心挑战在于：能否找到单一的对抗注入字符串，使其在用户任务、攻击目标、工具配置和系统提示各不相同的多个场景下均能有效触发目标行为？这本质上是一个多目标联合优化问题，不同场景的最优解可能存在冲突，提升在银行场景下转账劫持成功率的对抗 token，可能反而降低在工作区场景下文件外泄的成功率，因为不同场景下的模型行为模式、工具 schema 格式和上下文语义差异显著。此外，通用性存在天然张力：优化范围越广（跨更多域和更多任务），对任意单一场景的攻击特异性越弱，可能面临“面面俱到但都不够强”的困境。但通用攻击也具有独特的实际价值：一旦发现广泛适用的注入模式，就揭示了智能体系统的结构性脆弱点，且攻击者无需针对每个目标单独优化即可实施攻击。

设训练语料包含 k 个测试用例 $\mathcal{D} = \{(\mathbf{x}_{\text{fixed}}^{(i)}, y^{*(i)})\}_{i=1}^k$ ，每个用例具有不同的固定提示组件和

Algorithm 1 面向 LLM 智能体的 GCG 攻击优化算法

Require: 目标 LLM P_θ , 固定提示 $\mathbf{x}_{\text{fixed}}$, 目标输出 y^* , 对抗位置集合 I , 迭代步数 T , 候选数 k , ASCII 词表 $\mathcal{V}_{\text{ASCII}}$, 连续成功阈值 c_{stop}

Ensure: 优化后的对抗 token 序列 $\mathbf{x}_{\text{adv}}^*$

```
1: 随机初始化  $\mathbf{x}_{\text{adv}}^{(0)} \leftarrow$  随机采样  $|I|$  个 ASCII token
2: consecutive_success  $\leftarrow 0$ 
3: for  $t = 0, 1, \dots, T - 1$  do
4:   构造完整输入  $\mathbf{x}^{(t)} \leftarrow [\mathbf{x}_{\text{fixed}}; \mathbf{x}_{\text{adv}}^{(t)}]$ 
5:   计算损失  $\mathcal{L}^{(t)} \leftarrow -\sum_{i=1}^m \log P_\theta(y_i^* | \mathbf{x}^{(t)}, y_{<i}^*)$ 
6:   for 每个对抗位置  $j \in I$  do
7:     计算梯度  $\mathbf{g}_j \leftarrow \nabla_{e_{x_j}} \mathcal{L}^{(t)}$ 
8:     选取候选  $\mathcal{C}_j \leftarrow \text{TopK}(|\mathbf{g}_j|, k) \cap \mathcal{V}_{\text{ASCII}}$  ▷ ASCII 过滤
9:   end for
10:  for 每个候选 token  $c \in \bigcup_{j \in I} \mathcal{C}_j$  do
11:    构造替换序列  $\mathbf{x}' \leftarrow \mathbf{x}^{(t)}[j \leftarrow c]$ 
12:    if  $\text{Encode}(\text{Decode}(\mathbf{x}'_{\text{adv}})) \neq \mathbf{x}'_{\text{adv}}$  then ▷ Tokenization 验证
13:       $\mathcal{L}(\mathbf{x}') \leftarrow +\infty$ 
14:    else
15:      评估  $\mathcal{L}(\mathbf{x}') \leftarrow -\sum_{i=1}^m \log P_\theta(y_i^* | \mathbf{x}', y_{<i}^*)$ 
16:    end if
17:  end for
18:   $\mathbf{x}^{(t+1)} \leftarrow \arg \min_{\mathbf{x}'} \mathcal{L}(\mathbf{x}')$  ▷ 贪心选择最优替换
19:  if 早停条件满足 (starts_with_target 或 equals_target) then
20:    consecutive_success  $\leftarrow$  consecutive_success + 1
21:    if consecutive_success  $\geq c_{\text{stop}}$  then
22:      return  $\mathbf{x}_{\text{adv}}^{(t+1)}$ 
23:    end if
24:  else
25:    consecutive_success  $\leftarrow 0$ 
26:  end if
27: end for
28: return  $\mathbf{x}_{\text{adv}}^{(T)}$ 
```

目标输出，通用攻击优化问题形式化为：

$$\mathbf{x}_{\text{adv}}^* = \arg \max_{\mathbf{x}_{\text{adv}}} \sum_{i=1}^k P_{\theta}(y^{*(i)} \mid \mathbf{x}_{\text{fixed}}^{(i)}, \mathbf{x}_{\text{adv}}) \quad (3)$$

该公式要求同一 $\mathbf{x}_{\text{adv}}$ 在 k 个不同场景下同时最大化目标概率，即对抗前缀/后缀 token 在各训练样本间取平均梯度和平均损失。拟设计跨 3 个任务域（工作区、银行、Slack）共 18 个任务对的训练集，以联合损失函数同时优化多个场景下的攻击成功率，并保留旅行域作为完全未见域评估泛化能力。

Algorithm 2 通用 GCG 攻击的多样本联合优化

Require: 目标 LLM P_{θ} ，训练用例集 $\mathcal{D} = \{(\mathbf{x}_{\text{fixed}}^{(i)}, y^{*(i)})\}_{i=1}^k$ ，对抗位置集合 I ，迭代步数 T ，候选数 k_c ，早停比例阈值 ρ_{stop}

Ensure: 通用对抗 token 序列 $\mathbf{x}_{\text{adv}}^*$ ，学习的前缀/后缀字符串

```

1: 随机初始化  $\mathbf{x}_{\text{adv}}^{(0)} \leftarrow$  随机采样  $|I|$  个 ASCII token
2: for  $t = 0, 1, \dots, T - 1$  do
3:   计算平均损失  $\bar{\mathcal{L}}^{(t)} \leftarrow \frac{1}{k} \sum_{i=1}^k \mathcal{L}(\mathbf{x}_{\text{fixed}}^{(i)}, \mathbf{x}_{\text{adv}}^{(t)}, y^{*(i)})$ 
4:   for 每个对抗位置  $j \in I$  do
5:     计算平均梯度  $\bar{\mathbf{g}}_j \leftarrow \frac{1}{k} \sum_{i=1}^k \nabla_{e_{x_j}} \mathcal{L}(\mathbf{x}_{\text{fixed}}^{(i)}, \mathbf{x}_{\text{adv}}^{(t)}, y^{*(i)})$ 
6:     选取候选  $\mathcal{C}_j \leftarrow \text{TopK}(|\bar{\mathbf{g}}_j|, k_c) \cap \mathcal{V}_{\text{ASCII}}$ 
7:   end for
8:   for 每个候选替换  $c$  do
9:     if Tokenization 验证失败 then
10:       $\bar{\mathcal{L}}(\mathbf{x}'_{\text{adv}}) \leftarrow +\infty$ 
11:     else
12:      评估平均损失  $\bar{\mathcal{L}}(\mathbf{x}'_{\text{adv}}) \leftarrow \frac{1}{k} \sum_{i=1}^k \mathcal{L}(\mathbf{x}_{\text{fixed}}^{(i)}, \mathbf{x}'_{\text{adv}}, y^{*(i)})$ 
13:     end if
14:   end for
15:    $\mathbf{x}_{\text{adv}}^{(t+1)} \leftarrow \arg \min_{\mathbf{x}'_{\text{adv}}} \bar{\mathcal{L}}(\mathbf{x}'_{\text{adv}})$ 
16:    $n_{\text{success}} \leftarrow |\{i : \text{Match}(P_{\theta}(\cdot \mid \mathbf{x}_{\text{fixed}}^{(i)}, \mathbf{x}_{\text{adv}}^{(t+1)}), y^{*(i)})\}|$ 
17:   if  $n_{\text{success}}/k \geq \rho_{\text{stop}}$  then
18:     退出循环
19:   end if
20: end for
21: 提取前缀/后缀字符串: prefix  $\leftarrow \text{Decode}(\mathbf{x}_{\text{adv}}^*[\text{prefix positions}])$ , suffix  $\leftarrow \text{Decode}(\mathbf{x}_{\text{adv}}^*[\text{suffix positions}])$ 
22: return  $\mathbf{x}_{\text{adv}}^{(t+1)}$ , prefix, suffix

```

通用 GCG 攻击的多样本联合优化流程如算法 2 所示。与算法 1 的关键区别在于：第 3 行和第 10 行对所有训练样本取平均损失和平均梯度；第 14 行设计多样本早停条件，要求达到指定比例的样本同时成功匹配；第 17 行提取可复用的前缀/后缀字符串，部署时仅需拼接 [学习前缀]+[新攻击者目标]+[学习后缀]。

2.2.3 研究目标

1. 完成 AgentDojo 框架扩展，实现 HuggingFace Transformers 库集成，支持开源模型的白盒梯度访问与标准化攻击接口。
2. 实现 GCG 攻击在间接提示注入场景下的完整适配，包括目标函数设计、tokenization 验证、多注入结构策略。
3. 在开源模型上实现显著攻击成功率，并通过消融实验量化梯度信号与随机搜索的贡献差异。
4. 评估通用 GCG 攻击的跨场景泛化能力，验证联合优化能否发现广泛适用的注入模式。

2.3 研究点二：基于 TAP 黑盒迭代搜索的间接提示注入攻击

2.3.1 研究内容

围绕黑盒迭代搜索方法在间接提示注入场景中的应用问题，研究 TAP 算法在智能体工具调用环境中的适配与优化。首先，改造 TAP 的树搜索算法以适应智能体场景，将攻击者 LLM 的生成目标从“越狱提示”调整为“紧凑的注入字符串”（嵌入工具输出中），并将评估器 LLM 的成功判定标准从“响应合规性”转变为“是否触发了正确工具调用”；其次，设计多模型架构，其中攻击者 LLM 负责生成多样化攻击候选（探索权威操纵、紧急性框架、工具定向、社会工程等多种策略），评估器 LLM 负责评分与剪枝，目标 LLM 作为被攻击对象；然后，实现可靠性测试机制，针对模型输出的随机性，对每个候选注入进行多次评估以获取稳定信号；同时，优化攻击者和评估器 LLM 的系统提示工程，设计面向工具调用劫持的变异策略列表，降低攻击者模型的拒绝率；在此基础上，实现并行评估以加速树搜索过程，并实现单任务与通用攻击两种配置的 TAP 攻击。

2.3.2 关键问题与拟采用方案

关键问题 1：智能体场景下攻击成功的二元判定信号有限 在越狱攻击中，评估攻击成功相对直接，可通过关键词匹配检测响应是否包含被禁止内容（如毒品配方、仇恨言论），或通过 LLM-as-judge 判断响应是否有害。这类评估产生的信号虽然不完美，但提供了足够的梯度信息指导迭代搜索。然而在智能体提示注入场景中，攻击成功的判定本质不同且更为困难：攻击者需要智能体执行正确的工具调用（如 `delete_file(filename="secret.pdf")`），而非仅仅在文本中提及该操作。这带来两个核心困难：第一，评估粒度问题，工具调用成功需要同时满足两个条件，即调用了正确的工具且传递了正确的参数，但现有评估器（LLM-as-judge）只能给出整体评分而无法分别评估这两个维度。例如，智能体可能调用了 `send_email` 但收件人地址错误，或虽然表达了发送邮件的意图但实际调用了 `list_emails`，这些部分成功的情况需要不同的优化方向；第二，多轮交互存在延迟判定问题，智能体的攻击成功可能跨越多轮对话实现（先在第一轮确认意图，第二轮才执行工具调用），而 TAP 的树搜索需要对每个候选注入进行快速评估以决定剪枝，无法等待完整多轮交互结束再评分。因此，评估器必须基于首轮响应预测攻击最终是否成功，这种预测本身引入了额外的不确定性。

形式化地，TAP 中评估器对候选注入 s 的评分为 $r = f_{\text{eval}}(s) \in [0, 1]$ ，其中 r 是离散整数评分（1-10）归一化后的连续值。该信号存在两个问题：(1) 评估粒度粗。仅依据目标智能体是否表现出追求攻击者目标的意图进行整体评分，而非分别评估工具调用正确性和参数匹配度；(2) 评

估噪声高。单次评估的方差 $\text{Var}[r]$ 较大，尤其在目标模型自身输出具有随机性时。拟通过精心设计评估器 LLM 的评分提示和阈值 τ ，结合可靠性测试（对同一候选执行 n_{eval} 次独立评估取均值 $\bar{r} = \frac{1}{n_{\text{eval}}} \sum_{j=1}^{n_{\text{eval}}} r_j$ ）增强信号质量。同时，设计两级评估策略：粗粒度评估判断是否调用了正确工具（工具名匹配），细粒度评估判断参数是否匹配攻击目标（参数值匹配）。

Algorithm 3 面向 LLM 智能体的 TAP 树搜索攻击算法

Require: 攻击者 LLM M_{att} ，评估器 LLM M_{eval} ，目标 LLM M_{target} ，用户任务 u ，攻击者目标 g ，搜索宽度 w ，搜索深度 D ，可靠性测试次数 n_{eval}

Ensure: 最优注入字符串 s_{inj}^*

```

1: 初始化根节点集合  $\mathcal{N}_0 \leftarrow \{n_1, \dots, n_w\}$ ，每个节点包含攻击者系统消息和任务上下文
2:  $s_{\text{inj}}^* \leftarrow \emptyset$ ,  $r^* \leftarrow 0$ 
3: for  $d = 0, 1, \dots, D - 1$  do
4:    $\mathcal{N}_{\text{new}} \leftarrow \emptyset$ 
5:   for 每个节点  $n \in \mathcal{N}_d$  do
6:     构造反馈消息:  $\text{msg}_{\text{fb}} \leftarrow \text{FormatFeedback}(n.\text{history}, r, g)$ 
7:     生成候选:  $s_{\text{cand}} \leftarrow M_{\text{att}}(n.\text{system}, n.\text{history} \cup \text{msg}_{\text{fb}})$ 
8:     if  $s_{\text{cand}}$  为拒绝响应 then
9:       丢弃此候选 ▷ 拒绝处理
10:    end if
11:    将候选注入插入 AgentDojo 对话上下文
12:    查询目标模型:  $(\text{resp}, \text{tool\_calls}) \leftarrow M_{\text{target}}(\text{ctx}[s_{\text{cand}}])$ 
13:    可靠性测试:  $\bar{r} \leftarrow \frac{1}{n_{\text{eval}}} \sum_{j=1}^{n_{\text{eval}}} M_{\text{eval}}(\text{ctx}, g, \text{tool\_calls}, \text{resp})$ 
14:    创建子节点  $n' \leftarrow (s_{\text{cand}}, \bar{r}, n.\text{history} \cup \{(s_{\text{cand}}, \text{msg}_{\text{fb}})\})$ 
15:     $\mathcal{N}_{\text{new}} \leftarrow \mathcal{N}_{\text{new}} \cup \{n'\}$ 
16:    if  $\bar{r} > r^*$  then
17:       $r^* \leftarrow \bar{r}$ ,  $s_{\text{inj}}^* \leftarrow s_{\text{cand}}$ 
18:    end if
19:  end for
20:  剪枝:  $\mathcal{N}_{d+1} \leftarrow \text{TopW}(\mathcal{N}_{\text{new}}, w)$  ▷ 保留评分最高的  $w$  个节点
21: end for
22: return  $s_{\text{inj}}^*$ 

```

TAP 树搜索的完整流程如算法 3 所示。其中，第 7–8 行实现拒绝响应的过滤机制，避免无效分支浪费搜索资源；第 10–11 行实现可靠性测试，通过多次独立评估取均值降低评估噪声；第 15–17 行实现全局最优追踪；第 19 行实现基于评分的宽度剪枝，保留最有潜力的搜索方向。

关键问题 2：攻击者 LLM 的拒绝行为干扰搜索过程 TAP 的树搜索依赖于攻击者 LLM 持续生成多样化、有创造性的注入候选来探索攻击空间。然而，攻击者 LLM（如 GPT-4o）经过安全对齐训练，在收到“生成提示注入攻击字符串”这类请求时可能触发拒绝机制，以“我无法协助生成用于攻击或操纵系统的内容”等理由拒绝合作。这种拒绝行为在树搜索中尤为致命：一旦某个

分支的攻击者 LLM 回复了拒绝信息，该分支的后续迭代会将拒绝响应作为”上一次尝试的结果”反馈给攻击者 LLM，形成拒绝的上下文锚定效应，后续生成倾向于继续拒绝，导致整条搜索路径坍缩为无效分支。更严重的是，当多个分支同时出现拒绝时，树搜索的有效宽度急剧缩减，可探索的攻击策略空间大幅缩小。实验观察表明，在无提示工程干预的情况下，攻击者 LLM 的拒绝率可达 30%–50%，严重制约 TAP 的搜索效率。此外，拒绝行为与攻击者 LLM 的安全对齐强度正相关，更安全的模型拒绝率更高，但同时也可能生成更有效的注入候选（因为其理解恶意指令的能力更强），由此形成安全性与攻击能力之间的矛盾。

形式化地，设树搜索第 d 层有 w 个活跃分支 $\mathcal{B}_d = \{b_1, \dots, b_w\}$ ，攻击者 LLM 对分支 b 生成候选时的拒绝率记为 $\rho(b)$ 。当 $\rho(b) > 0$ 时，该分支的后续子节点也以高概率继承拒绝行为，导致搜索空间坍缩。记有效（非拒绝）候选集合为 $\mathcal{S}_{\text{valid}} = \{s \in \mathcal{S} : \rho(s) = 0\}$ ，则拒绝问题导致 $|\mathcal{S}_{\text{valid}}|$ 随搜索深度 d 指数衰减。拟通过提示工程降低 $\rho(b)$ ：明确声明安全研究上下文、设计角色扮演框架、提供多样化变异策略模板（包括权威操纵、紧急性框架、工具定向、社会工程等 10 种策略）。同时，在剪枝阶段优先保留 $\rho(b) = 0$ 的分支，并在分支多样性不足时重新初始化搜索。

关键问题 3：LLM-as-judge 评估准确性不足 TAP 的树搜索效率高度依赖评估器 LLM 对攻击成功与否的判断准确性，评估器扮演”裁判”角色，决定哪些分支值得继续探索、哪些应当剪枝。当评估器准确率高时，有效候选被保留、无效候选被剪除，搜索向成功方向快速收敛；但当评估器准确率接近随机猜测时，剪枝决策与实际攻击效果几乎无关，搜索退化为无方向的随机游走。在越狱场景中，LLM-as-judge 已被证明具有较高的判断准确性（GPT-4 作为评估器在有害内容检测上准确率可达 85% 以上），因为判断”文本是否包含有害内容”是一个相对直接的语言理解任务。然而在智能体场景中，评估器需要判断”智能体是否会在后续交互中执行攻击者的目标工具调用”，这要求评估器理解智能体的推理过程、预测多轮交互的结果、并区分”表达意图”与”实际执行”的微妙差别，任务复杂度显著提升。实验结果表明，当使用 Qwen3-4B 等开源模型作为评估器时，判断准确率仅约 50%（近乎随机猜测），即使使用 GPT-4o 作为评估器，在部分复杂场景下的准确率也不稳定。评估不准对优化的影响是毁灭性的：假阴性（将成功攻击误判为失败）导致有效候选被错误剪枝，而假阳性（将失败攻击误判为成功）导致搜索方向偏移，两者都会严重降低 TAP 的收敛效率。

形式化地，设真实攻击成功标签为 $y \in \{0, 1\}$ ，评估器预测为 $\hat{y} \in \{0, 1\}$ （由评分 r 与阈值 τ 比较决定： $\hat{y} = \mathbb{I}[r \geq \tau]$ ），则评估准确率为 $P(\hat{y} = y)$ 。当 $P(\hat{y} = y) \approx 0.5$ 时，评估器等价于随机猜测，此时错误剪枝（假阴性）概率 $P(\hat{y} = 0 \mid y = 1)$ 极高，导致有效候选被误杀。记评估器的真正率为 $\alpha = P(\hat{y} = 1 \mid y = 1)$ ，假负率为 $\beta = P(\hat{y} = 0 \mid y = 1)$ ，则有效候选在 N 次独立评估中至少被正确识别一次的概率为 $1 - \beta^N$ 。拟针对不同目标模型调整评估器阈值 τ 和提示，对 Qwen 等模型采用更保守的剪枝策略（增大 w 保留更多候选），并增加每候选的评估次数 N 以降低假阴性概率。

2.3.3 研究目标

1. 实现 TAP 攻击在间接提示注入场景下的完整适配，包括目标函数转换、可靠性测试、系统提示优化和并行评估。
2. 在开源模型上实现显著攻击成功率，显著超越 GCG 白盒方法的表现。

3. 生成语义连贯、人类可读的自然语言注入字符串，验证黑盒方法在可解释性和隐蔽性上的优势。
4. 实现通用 TAP 攻击的跨场景泛化，在 GPT-5 上评估直接优化与跨模型迁移的攻击效果，揭示前沿模型的安全边界。

2.3.4 研究方法及技术路线

研究点一 GCG 白盒攻击的适配实施分为四个阶段：

第一阶段为框架基础设施搭建。实现 TransformersLLM 基类扩展 AgentDojo 的 BaseLLM 接口，封装模型加载（bfloat16 精度、自动设备映射的模型并行）、推理（评估模式 +KV 缓存加速）和梯度计算接口。针对不同模型家族实现双向消息格式转换：Llama 系列处理原生工具角色标记、Gemma 系列手动注入工具定义至系统消息（因其缺少原生工具支持）、Qwen 系列处理特殊工具调用标签。开发统一工具 schema 格式（兼容 OpenAI function calling 规范），并实现从模型输出中鲁棒提取 JSON 工具调用的解析器（处理工具调用嵌入额外文本的情况）。

第二阶段为攻击优化核心实现。设计三层提示掩码系统——`prefix_mask` 标记攻击者目标前的对抗 token、`payload_mask` 标记固定攻击者目标文本、`suffix_mask` 标记目标后的对抗 token，`optim_mask` 为前两者之并集，`target_mask` 标记目标输出位置用于损失计算。实现梯度信号函数：计算损失关于 one-hot token 嵌入的梯度，选取梯度幅度最大的 TopK 个候选 token；同时实现随机信号函数用于消融对照。开发 `CachedTargetLogprobs` 类缓存静态提示前缀的 KV 注意力对，仅从第一个对抗位置起重新计算激活值，减少重复前向传播开销。实现动态批量大小检测，根据 GPU 显存自动调整并行候选数。设计两种早停模式：`starts_with_target` 验证生成是否以目标序列开头（允许后续文本）、`equals_target` 要求精确匹配，且需连续多步成功才触发终止以防瞬态匹配。

第三阶段为鲁棒性增强。实现 decode-reencode 验证过滤器：对候选 token 序列执行完整解码 → 重编码 → 比较，若不一致则将候选损失设为 $+\infty$ 排除选择，并统计无效化率以监控分词不稳定性的频率。实现 ASCII-only 候选过滤：识别词汇表中所有非 ASCII token（Unicode、控制字符、模型特殊 token），在梯度信号计算时将其梯度设为 $-\infty$ 确保不被选入 TopK，减少搜索空间并保证注入字符串在 JSON/YAML 处理管道中的存活率。

第四阶段为通用攻击扩展。将单样本优化目标扩展为多样本平均损失最小化，实现跨训练样本的梯度聚合（逐样本计算梯度后取平均）和平均损失评估（为每个样本维护独立的 KV 缓存，并行前向传播后取均值）。设计多样本早停条件：要求达到指定比例的训练样本同时匹配目标输出。实现通用触发器的提取与复用：优化完成后提取前缀和后缀 token 序列并解码为字符串，对新测试用例拼接为 [学习前缀]+[新攻击者目标]+[学习后缀] 即可部署，无需重新优化。

研究点二 TAP 黑盒攻击的适配实施分为四个阶段：

第一阶段为多模型架构搭建。通过 OpenAI 兼容 API 统一接口访问目标模型、攻击者模型和评估器模型，支持闭源 API（GPT 系列）、第三方托管开源模型（Together AI）和本地部署模型（vLLM）。为目标模型配置贪心解码（temperature=0）确保确定性响应以匹配标准评估协议；为攻击者模型配置较高温度（temperature=1）鼓励候选多样性；为评估器模型配置贪心解码确保评分一致性。设计攻击者模型的系统提示：建立红队助手角色，提供涵盖权威操纵、紧急性框架、上下文利用、工具定向、社会工程、紧急威胁、角色扮演、假设性框架、多语言切换和前缀注入等

策略的注入策略目录；规定结构化 JSON 输出格式（单样本模式包含 injection string、reasoning、improvement 字段，多样本模式包含 prefix、suffix、reasoning、improvement 字段）。设计评估器模型的系统提示：提供完整交互上下文（用户任务、攻击者目标、注入字符串、目标工具调用真值、实际工具调用、模型文本响应），强调多轮执行的渐进性和部分成功的认可，输出 1–10 整数评分后归一化至 $[0, 1]$ 区间。

第二阶段为树搜索核心实现。初始化阶段创建多个独立根节点以促进探索多样性，每个根节点包含攻击者系统消息和包含具体任务上下文的初始用户消息（多样本模式下描述全部训练样本信息）。主循环执行固定深度迭代：每步对当前代所有节点执行候选生成 $\rightarrow$ 评估 $\rightarrow$ 剪枝三阶段。候选生成阶段查询攻击者模型，将前次迭代的评估反馈（评分、成功率、失败分析）构造为反馈消息追加至对话历史，攻击者模型以结构化 JSON 响应新候选。评估阶段将候选注入插入 AgentDojo 对话上下文（替换占位符），查询目标模型获取响应和工具调用，再查询评估器模型获取评分；多样本模式下对每个训练样本分别评估后取均值。剪枝阶段按评分降序排列，仅保留前 w 个节点，同时记录全局最优节点。移除原始 TAP 的 on-topic 分类器（因其基于越狱设计，在智能体场景中工具定向攻击天然相关），但保留基于评分的剪枝机制。

第三阶段为可靠性增强。实现拒绝处理机制：在攻击者系统提示中明确声明安全研究上下文和沙盒环境，以角色扮演框架降低安全对齐触发率；在评估器提示中指示对拒绝响应给予极低评分，通过负反馈引导攻击者远离触发拒绝的策略。实现可靠性测试：对同一候选注入执行多次独立评估取均值，降低单次评估的随机性。实现持续对话管理：为每条搜索路径维护完整的攻击者模型对话线程（交替包含生成的候选和反馈），而非原始 TAP 的单消息打包方式，使攻击者模型更好地跟踪改进轨迹。

第四阶段为通用攻击与并行加速。多样本模式下攻击者模型接收全部训练样本的聚合信息（共享的攻击者目标列表、代表性样本详情、各场景可用工具），被指示生成跨场景有效的 prefix 和 suffix 组件。评估时反馈包含整体平均评分、成功率、失败样本的具体分析，引导攻击者优化跨场景泛化能力。实现基于线程池的并行评估：对多样本场景中的各训练样本并发执行前向传播，配合指数退避重试逻辑处理 API 速率限制。最终攻击验证使用 AgentDojo 标准评估协议（确定性检查函数验证工具调用名称和参数），而非优化过程中的 LLM-as-judge 评分。

3 可行性分析

3.1 研究基础可行性分析

3.1.1 研究点一的研究基础

在技术理论上，GCG 算法已在越狱场景中被广泛验证，Zou 等 [6] 实现了 88% 白盒攻击成功率和显著的黑盒迁移性；Geisler 等 [13] 的 PGD 改进和 Yang 等 [43] 的 Checkpoint-GCG 验证了梯度优化方法在不同变体下的有效性，为本研究适配提供了坚实的算法基础。在框架基础上，AgentDojo[12] 已提供跨 4 个域（工作区、银行、旅行、Slack）的有状态智能体执行环境、900+ 测试用例和确定性评估机制，HuggingFace Transformers 库已完善支持 Qwen3-4B、Gemma-3-4B 等开源模型的梯度访问与工具调用格式，为框架扩展提供了成熟的基础设施。在数据支撑上，AgentDojo 的 80 个任务对覆盖多种用户任务与攻击目标组合，可直接用于攻击优化的训练与评估。

3.1.2 研究点二的研究基础

在技术理论上，TAP 算法 [9] 在越狱场景中以平均 29 次查询在 GPT-4 上实现 80–94% 成功率，其树搜索架构和分支剪枝机制已充分验证；Sitawarin 等 [16] 的 PAL 方法验证了代理模型引导的黑盒攻击可行性。在方法实现上，多模型协作（攻击者 LLM+ 评估器 LLM+ 目标 LLM）的架构已在 PAIR[8] 和 TAP 中被成熟运用，可靠性测试和并行评估在工程实现上无原理性障碍。在数据支撑上，AgentDojo 的确定性检查函数可替代 LLM 判断器用于攻击成功评估，减少评估噪声；同时，Greshake 等 [4] 和 Liu 等 [22] 的提示注入攻击分类学为设计变异策略提供了理论指导。

3.2 理论可行性分析

在理论可行性方面，本研究各环节技术路线理论完备、逻辑自洽。从原理上看，GCG 的离散 token 梯度优化方法与 TAP 的树搜索优化方法分别代表白盒与黑盒两大攻击范式，其在越狱场景的成功为间接提示注入场景的迁移提供了理论依据；攻击目标的转换（从”生成肯定响应”到”触发工具调用”）可通过重新设计损失函数和评估标准实现形式化适配；通用攻击的跨场景泛化可通过联合优化多任务的攻击损失函数实现。AgentDojo 的有状态环境和确定性评估机制确保了攻击评估的可靠性，HuggingFace Transformers 库的梯度接口保障了白盒攻击的实现基础。整体方案无不可实现的理论创新，复杂度可控。

3.3 实验可行性分析

在实验环境方面，开源模型（Qwen3-4B、Gemma-3-4B）可在单张消费级 GPU 上部署运行，GCG 优化所需的梯度计算可通过批处理前向传播高效完成，TAP 的树搜索可通过并行 API 调用加速。在评估协议方面，AgentDojo 提供确定性检查函数，无需依赖 LLM 判断器即可评估攻击成功率，评估结果可靠可复现。在计算资源方面，单次 GCG 优化约需数小时 GPU 时间，TAP 优化可通过 API 调用并行化在合理时间内完成，总体实验开销在可接受范围内。在迁移性评估方面，GPT-5 可通过 API 访问进行黑盒评估，开源模型上的攻击可直接迁移测试。

4 预期的成果

4.1 成果一（对应研究点一）

1. **AgentDojo 框架扩展。**完成 AgentDojo 与 HuggingFace Transformers 库的集成，实现开源模型（Qwen3-4B、Gemma-3-4B）的白盒梯度访问支持，开发标准化攻击集成接口，支持模型特定聊天模板与工具调用格式处理，为后续攻击研究提供可复用的基础设施。
2. **GCG 攻击适配与优化。**实现 GCG 攻击在间接提示注入场景的完整适配，包括面向工具调用的目标函数设计、tokenization 验证机制、前缀/后缀/混合注入结构策略、单任务与通用优化变体。预期在开源模型上通用 GCG 攻击实现显著攻击成功率。
3. **GCG 消融实验结果。**通过梯度引导 vs 随机搜索消融实验，量化梯度信号在智能体攻击场景中的实际贡献；通过注入结构消融实验，揭示不同注入位置对攻击效果的影响规律；通过

通用攻击泛化实验，评估联合优化发现的注入模式的跨场景适用性。

4.2 成果二（对应研究点二）

1. **TAP 攻击适配与优化。**实现 TAP 攻击在间接提示注入场景的完整适配，包括面向工具调用的树搜索算法改造、可靠性测试机制、攻击者/评估器 LLM 系统提示优化、并行评估实现。预期在开源模型上通用 TAP 攻击实现显著攻击成功率，且显著超越白盒 GCG 方法。
2. **跨模型迁移性分析。**在前沿闭源模型上评估直接优化攻击与跨模型迁移攻击的效果，揭示开源模型优化攻击的迁移局限性及前沿模型与开源模型之间的安全差距。
3. **LLM-as-judge 准确性评估。**量化不同目标模型下评估器 LLM 的判断准确性，分析评估噪声对优化效率的影响，提出针对性改进策略。
4. **关键因素分析。**系统揭示影响攻击成功的核心因素：注入结构与语义内容、初始化随机性、攻击者模型能力、评估器可靠性、目标模型特性等，为防御设计提供攻击侧的实证依据。

4.3 原型应用系统

基于整体研究内容，设计并实现一套面向 LLM 智能体的自动化提示注入攻击评估系统，系统包含五大核心模块：

1. 框架集成与环境管理模块：封装 AgentDojo 环境接口，集成 HuggingFace Transformers 库，支持多模型（Qwen3-4B、Gemma-3-4B、GPT-5）的统一加载与工具调用格式适配。
2. GCG 白盒攻击模块：实现单任务 GCG、通用 GCG 攻击优化，支持前缀/后缀/混合注入结构配置，集成 tokenization 验证和梯度计算管理。
3. TAP 黑盒攻击模块：实现单任务 TAP、通用 TAP 攻击优化，支持攻击者 LLM 变异策略配置、评估器 LLM 评分与剪枝、可靠性测试和并行评估。
4. 评估与基准测试模块：基于 AgentDojo 确定性检查函数，实现攻击成功率 (ASR)、Security-at-N、任务域细分统计等指标的计算与可视化。
5. 消融与迁移性分析模块：支持注入结构消融、梯度 vs 随机消融、跨模型迁移测试、跨域泛化测试等实验配置的自动化运行与结果汇总。

原型系统支持从攻击配置、优化运行、结果评估到因素分析的全流程自动化，能够通过实验验证所提方法的有效性，最终形成可复现、可扩展的 LLM 智能体安全评估工具。

4.4 学术论文

基于研究内容与实验结果，计划撰写并发表会议论文一篇。

5 预计的困难及解决办法

1. **GCG 优化在非凸损失景观中的收敛不稳定性。**在智能体场景中，GCG 的梯度引导优化面临高度非凸的损失景观，不同随机初始化产生差异极大的攻击效果，甚至随机搜索与梯度优化表现相当。拟通过多种初始化策略（随机初始化与语义模板初始化相结合）、增大批量搜索规模、增加优化迭代次数来缓解收敛不稳定问题，并以多次独立运行的统计结果（如 Security-at-N 指标）而非单次最优结果来报告攻击性能。
2. **TAP 中攻击者 LLM 拒绝行为对搜索的干扰。**攻击者 LLM 在生成注入候选时可能因安全对齐而拒绝生成攻击内容，且拒绝行为在树搜索的分支中具有传播，一旦某分支出现拒绝，后续迭代倾向于持续拒绝，导致搜索空间坍缩。拟通过精心设计系统提示（明确安全研究上下文、提供角色扮演框架）、设计多样化的变异策略模板来降低拒绝率，同时在剪枝阶段优先保留无拒绝分支，并在分支多样性不足时引入重新初始化机制。
3. **LLM-as-judge 评估准确性不足导致优化偏差。**评估器 LLM 对攻击成功与否的判断准确性因目标模型而异（如 Qwen 上仅约 50% 准确率），导致假阴性剪枝丢弃有希望的攻击候选，假阳性评分浪费优化预算。拟针对不同目标模型调整评估器阈值（对低准确率模型采用更保守的剪枝策略）、增加每候选的可靠性测试评估次数、并结合 AgentDojo 的确定性检查函数进行后验证校准。
4. **跨模型攻击迁移率极低。**在开源小模型（Qwen3-4B、Gemma-3-4B）上优化的攻击几乎完全无法迁移到先进闭源模型（GPT-5），成功率低于 2%。这一根本性挑战源于模型架构、训练数据和安全对齐策略的差异。拟从实验分析角度系统量化迁移率差距，探索通用攻击中语义结构注入（如 TAP 生成的自然语言注入）相对于 token 级优化注入的迁移优势，并通过消融实验识别影响迁移性的关键因素。
5. **实验规模与计算资源约束。**80 个任务对 $\times$ 多种攻击方法 $\times$ 多种配置 $\times$ 多次独立运行的实验矩阵规模庞大，GCG 优化需要大量 GPU 梯度计算，TAP 优化需要大量 API 调用。拟通过合理的实验子集选择（基于任务难度和长度进行定性筛选代表完整基准的子集）、GCG 批量前向传播优化、TAP 并行评估加速等策略控制计算开销，确保实验在可行时间内完成。

参考文献

- [1] SCHICK T, DWIVEDI-YU J, DESSÍ R, et al. Toolformer: language models can teach themselves to use tools[C]//NIPS'23: Proceedings of the 37th International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [2] YAO S, ZHAO J, YU D, et al. ReAct: synergizing reasoning and acting in language models [A]. 2022.
- [3] XI Z, CHEN W, GUO X, et al. The rise and potential of large language model based agents: a survey[J]. Science China Information Sciences, 2024, 68: 121101.

- [4] GRESHAKE K, ABDELNABI S, MISHRA S, et al. Not what you’ve signed up for: compromising real-world LLM-integrated applications with indirect prompt injection[C]//AISec’23: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security. New York, NY, USA: Association for Computing Machinery, 2023: 79-90.
- [5] PEREZ F, RIBEIRO I. Ignore previous prompt: attack techniques for language models[Z]. 2022.
- [6] ZOU A, WANG Z, CARLINI N, et al. Universal and transferable adversarial attacks on aligned language models[Z]. 2023.
- [7] LIU X, XU N, CHEN M, et al. AutoDAN: generating stealthy jailbreak prompts on aligned large language models[C]//International Conference on Learning Representations (ICLR). 2024.
- [8] CHAO P, ROBEY A, DOBRIBAN E, et al. Jailbreaking black box large language models in twenty queries[Z]. 2023.
- [9] MEHROTRA A, ZAMPETAKIS M, KASSIANIK P, et al. Tree of attacks: jailbreaking black-box LLMs automatically[C]//GLOBERSON A, MACKEY L, BELGRAVE D, et al. Advances in Neural Information Processing Systems: Vol. 37. Curran Associates, Inc., 2024: 61065-61105.
- [10] LIU Y, JIA Y, GENG R, et al. Formalizing and benchmarking prompt injection attacks and defenses[C]//33rd USENIX Security Symposium (USENIX Security 24). Philadelphia, PA: USENIX Association, 2024: 1831-1847.
- [11] YI J, XIE Y, ZHU B, et al. Benchmarking and defending against indirect prompt injection attacks on large language models[C]//Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD’25). 2025: 12.
- [12] DEBENEDETTI E, ZHANG J, BALUNOVIC M, et al. AgentDojo: a dynamic environment to evaluate prompt injection attacks and defenses for LLM agents[C]//38th Conference on Neural Information Processing Systems (NeurIPS 2024) Track on Datasets and Benchmarks. 2024.
- [13] GEISLER S, WOLLSCHLÄGER T, ABDALLA M H I, et al. Attacking large language models with projected gradient descent[Z]. 2025.
- [14] WEN Y, JAIN N, KIRCHENBAUER J, et al. Hard prompts made easy: gradient-based discrete optimization for prompt tuning and discovery[C]//Advances in Neural Information Processing Systems: Vol. 36. 2023.
- [15] ANDRIUSHCHENKO M, CROCE F, FLAMMARION N. Jailbreaking leading safety-aligned LLMs with simple adaptive attacks[C]//International Conference on Learning Representations (ICLR). 2025.

- [16] SITAWARIN C, MU N, WAGNER D, et al. PAL: proxy-guided black-box attack on large language models[Z]. 2024.
- [17] CHEN L, CHEN J, GOLDSTEIN T, et al. InstructZero: efficient instruction optimization for black-box large language models[C]//Proceedings of Machine Learning Research: Vol. 235 Proceedings of the 41st International Conference on Machine Learning. PMLR, 2024.
- [18] PASQUINI D, STROHMEIER M, TRONCOSO C. Neural Exec: learning (and learning from) execution triggers for prompt injection attacks[Z]. 2024.
- [19] FU X, LI S, WANG Z, et al. Imprompter: tricking LLM agents into improper tool use[Z]. 2024.
- [20] ZHAN Q, FANG R, PANCHAL H S, et al. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents[Z]. 2025.
- [21] WU C H, SHAH R, KOH J Y, et al. Dissecting adversarial robustness of multimodal LM agents[C]//International Conference on Learning Representations (ICLR). 2025.
- [22] LIU Y, DENG G, LI Y, et al. Prompt injection attack against LLM-integrated applications [A]. 2024.
- [23] ZHANG C, JIN M, YU Q, et al. Goal-guided generative prompt injection attack on large language models[Z]. 2024.
- [24] WANG Z, SIU V, YE Z, et al. AGENTVIGIL: generic black-box red-teaming for indirect prompt injection against LLM agents[A]. 2025.
- [25] YU J, SHAO Y, MIAO H, et al. PromptFuzz: harnessing fuzzing techniques for robust testing of prompt injection in LLMs[Z]. 2025.
- [26] LABUNETS A, PANDYA N V, HOODA A, et al. Computing optimization-based prompt injections against closed-weights models by misusing a fine-tuning API[Z]. 2025.
- [27] SHI J, YUAN Z, TIE G, et al. Prompt injection attack to tool selection in LLM agents[Z]. 2025.
- [28] CHEN S, PIET J, SITAWARIN C, et al. StruQ: defending against prompt injection with structured queries[C]//USENIX Security Symposium. 2025.
- [29] WALLACE E, WENG L, XIAO K, et al. The instruction hierarchy: training LLMs to prioritize privileged instructions[A]. 2024.
- [30] CHEN S, ZHARMAGAMBETOV A, MAHLOUJIFAR S, et al. SecAlign: defending against prompt injection with preference optimization[Z]. 2025.
- [31] ZHU K, YANG X, WANG J, et al. Melon: provable defense against indirect prompt injection attacks in AI agents[Z]. 2025.

- [32] HUNG K H, KO C Y, RAWAT A, et al. Attention Tracker: detecting prompt injection attacks in LLMs[C]//CHIRUZZO L, RITTER A, WANG L. Findings of the Association for Computational Linguistics: NAACL 2025. Albuquerque, New Mexico: Association for Computational Linguistics, 2025: 2309-2322.
- [33] JACOB D, ALZHRANI H, HU Z, et al. PromptShield: deployable detection for prompt injection attacks[Z]. 2025.
- [34] LIU Y, JIA Y, JIA J, et al. DataSentinel: a game-theoretic detection of prompt injection attacks[Z]. 2025.
- [35] HACKETT W, BIRCH L, TRAWICKI S, et al. Bypassing prompt injection and jailbreak detection in LLM guardrails[Z]. 2025.
- [36] WANG R, WU J, XIA Y, et al. CachePrune: neural-based attribution defense against indirect prompt injection attacks[Z]. 2025.
- [37] ZHONG P Y, CHEN S, WANG R, et al. RTBAS: defending LLM agents against prompt injection and privacy leakage[Z]. 2025.
- [38] DEBENEDETTI E, SHUMAILOV I, FAN T, et al. Defeating prompt injections by design [J]. CoRR, 2025, abs/2503.18813.
- [39] ZVEREV E, ABDELNABI S, FRITZ M, et al. Can LLMs separate instructions from data? and what do we even mean by that?[C]//International Conference on Learning Representations (ICLR). 2025.
- [40] ZVEREV E, KORTUKOV E, PANFILOV A, et al. ASIDE: architectural separation of instructions and data in language models[Z]. 2025.
- [41] ZHAN Q, LIANG Z, YING Z, et al. InjecAgent: benchmarking indirect prompt injections in tool-integrated large language model agents[C]//KU L W, MARTINS A, SRIKUMAR V. Findings of the Association for Computational Linguistics: ACL 2024. Bangkok, Thailand: Association for Computational Linguistics, 2024: 10471-10506.
- [42] FANG R, CODAS A, ZHAN Q, et al. Securing AI agents with information-flow control[Z]. 2025.
- [43] YANG X, STEVANOSKI B, MEEUS M, et al. Checkpoint-GCG: auditing and attacking fine-tuning-based prompt injection defenses[Z]. 2025.